TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 2

Pods, YAML and kubectl

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 2: Pods, YAML and kubectl 3

2.1 What is a pod? 3

2.2 Imperative vs declarative 4

2.3 Anatomy of a YAML manifest 4

2.4 Multi-container patterns 5

2.5 Pod lifecycle 6

2.6 Labels, selectors and annotations 7

2.7 Live lab: the kubectl toolbox 7

Class 2 - Top 20 interview questions 9

K8S · CLASS 02 · 60-90 minutes

Pods, YAML and kubectl

By the end you can write pod YAML from memory, run multi-container pods, and use labels, selectors and the most important kubectl commands.

What you will learn today

- What a pod is and why Kubernetes does not run bare containers
- Imperative commands vs declarative YAML
- The four required fields of every manifest
- Multi-container pods: sidecar pattern, shared volume, init containers
- Pod lifecycle phases and restart policies
- Labels, selectors and annotations
- Lab: kubectl get, describe, logs, exec, port-forward, explain, dry-run

Part	Topic	Time
1	Story and what a pod is	10 min
2	YAML anatomy and kubectl basics	20 min
3	Multi-container pods and lifecycle	20 min
4	Labels and selectors	10 min
5	Live lab, recap and interview questions	25 min

2.1 What is a pod?

Ravi's story
Ravi wanted to run the orders app together with a small log-shipper container that reads the app's log files and sends them to the monitoring system. In Docker he had to set up a shared volume and network by hand. His senior said: "In Kubernetes, put both in one **pod** - they automatically share network and storage."

A **pod** is the smallest deployable unit in Kubernetes. It wraps one or more containers that:

- share **one IP address** and port space - they talk to each other on `localhost`;
- can share **volumes** (folders);
- are always **scheduled together** on the same node and live and die together.

Everyday example
A pod is like a **flat** in an apartment building. The flat has one postal address (the IP). Flatmates (containers) share the kitchen and living room (network and volumes) and can shout to each other directly (localhost). If the flat is demolished, everyone moves out together.

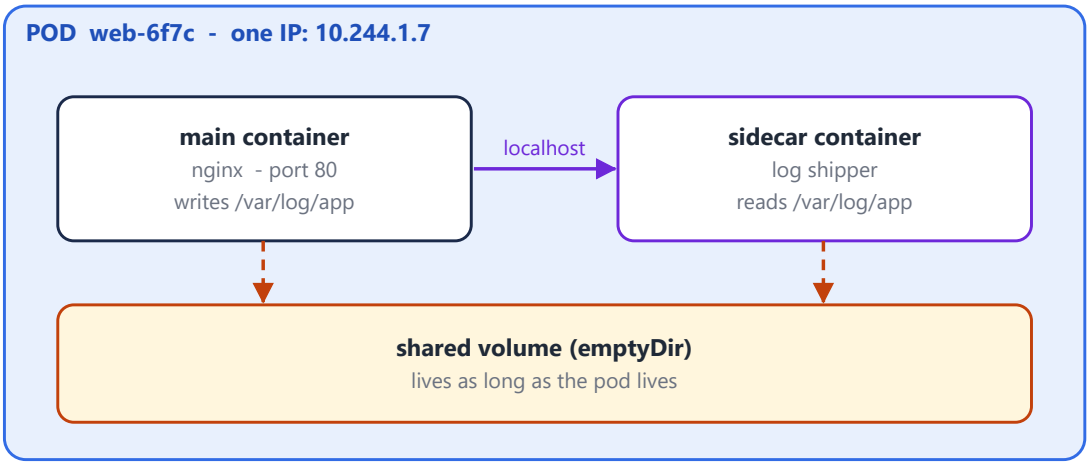


Figure 2.1 - A pod with a main container and a sidecar sharing IP and a volume

Remember
Most pods have exactly one container. Add a second container only when it must live and scale with the first one (log shipper, proxy, config reloader).

2.2 Imperative vs declarative

Style	Example	Use it for
Imperative command	`kubectl run web --image=nginx:1.27`	Quick tests, learning, exams
Imperative with file	`kubectl create -f pod.yaml`	One-time creation; fails if it exists
Declarative	`kubectl apply -f pod.yaml`	Real projects: files live in Git, apply is repeatable

2.3 Anatomy of a YAML manifest

Every Kubernetes object has the same four top-level fields:

Field	Meaning	Example
apiVersion	Which API group/version defines this object	`v1`, `apps/v1`, `networking.k8s.io/v1`
kind	The type of object	`Pod`, `Deployment`, `Service`
metadata	Name, namespace, labels, annotations	`name: web`
spec	The desired state - different for every kind	containers, replicas, ports...

pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: web
    tier: frontend
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

Kubernetes adds a fifth section, **status**, which you never write - it is the actual state reported by the cluster. See it with `kubectl get pod web -o yaml`.

Pro tip

- Do not memorise YAML - generate it: `kubectl run web --image=nginx:1.27 --dry-run=client -o yaml > pod.yaml`
- Forgot a field name? `kubectl explain pod.spec.containers` and `kubectl explain pod.spec --recursive` are built-in documentation.

Common mistake

- YAML uses **spaces only** - a single tab breaks the file.
- Indentation is meaning: `ports` must be inside the container item, not beside `containers`.
- Using `latest` image tags makes deployments unpredictable - always pin a version.

2.4 Multi-container patterns

Pattern	What the extra container does	Example
Sidecar	Adds a helper feature alongside the main app	Log shipper, service-mesh proxy (Envoy)
Ambassador	Proxies outbound connections for the app	Local proxy to a remote database
Adapter	Converts the app's output into a standard format	Exporter turning logs into metrics
Init container	Runs to completion **before** app containers start	Wait for DB, download config, run migrations

sidecar.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: web-with-sidecar
spec:
  volumes:
    - name: logs
      emptyDir: {}
  initContainers:
    - name: wait-a-bit
      image: busybox:1.36
      command: ["sh", "-c", "echo preparing...; sleep 5"]
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while true; do date >> /logs/app.log; sleep 2; done"]
      volumeMounts:
        - name: logs
          mountPath: /logs
    - name: log-shipper
      image: busybox:1.36
      command: ["sh", "-c", "tail -F /logs/app.log"]
      volumeMounts:
        - name: logs
          mountPath: /logs
```

Did you know?
Since Kubernetes 1.33, **native sidecars** are stable: an init container with `restartPolicy: Always` starts before the app and keeps running beside it. This fixes old problems such as Jobs never finishing because the sidecar kept running.

2.5 Pod lifecycle

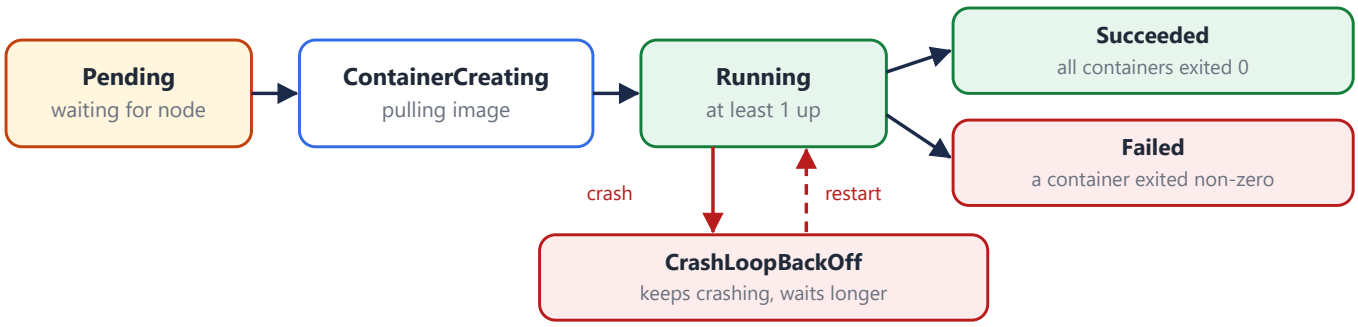


Figure 2.2 - Pod phases and the CrashLoopBackOff state

Phase / state	Meaning
Pending	Accepted, but not running yet - waiting for scheduling or image download.
Running	Bound to a node and at least one container is running.
Succeeded	All containers exited with code 0 and will not restart (typical for Jobs).
Failed	All containers stopped and at least one exited with an error.
Unknown	The node stopped reporting, so the state cannot be read.

Phase / state	Meaning
CrashLoopBackOff	Not a phase but a waiting reason: the container keeps crashing and Kubernetes waits longer (10s, 20s, 40s ... up to 5 min) before each restart.

The pod's `restartPolicy` decides what the kubelet does when a container exits: **Always** (default, for long-running apps), **OnFailure** (Jobs), or **Never**.

2.6 Labels, selectors and annotations

Labels are key-value tags on objects, such as `app=orders`, `env=prod`, `version=v2`. **Selectors** find objects by label. This is how a Deployment knows which pods it owns and how a Service knows where to send traffic - not by name, but by label.

> **Everyday example**
> In a supermarket, items carry stickers: "organic", "discount", "dairy". The store manager can say "move all items with discount AND dairy to aisle 3" without knowing each product name. Labels are the stickers; selectors are the manager's instruction.

PowerShell

```
kubectl get pods --show-labels
kubectl get pods -l app=web                    # equality
kubectl get pods -l 'env in (dev,qa)'          # set-based
kubectl label pod web env=dev                  # add a label
kubectl label pod web env-                     # remove it
```

Annotations are also key-value pairs but are *not* used for selection. They hold extra information for tools and humans: build number, Git commit, contact e-mail, or settings read by an ingress controller.

2.7 Live lab: the kubectl toolbox

PowerShell

```
kubectl apply -f pod.yaml
kubectl get pods -o wide                # IP and node
kubectl describe pod web                # full details + Events at the bottom
kubectl logs web                        # container output
kubectl exec -it web -- sh              # shell inside (type exit to leave)
kubectl port-forward pod/web 8080:80    # open http://localhost:8080 in a browser

kubectl apply -f sidecar.yaml
kubectl get pod web-with-sidecar -w     # watch Init:0/1 -> PodInitializing -> Running
kubectl logs web-with-sidecar -c log-shipper -f
kubectl exec -it web-with-sidecar -c app -- cat /logs/app.log

kubectl delete -f pod.yaml -f sidecar.yaml
```

Now break things on purpose and read the errors:

PowerShell

```
kubectl run broken --image=nginx:doesnotexist
kubectl get pods            # ErrImagePull, then ImagePullBackOff
kubectl describe pod broken # read the Events section

kubectl run crashy --image=busybox:1.36 -- sh -c "exit 1"
kubectl get pods -w         # Error -> CrashLoopBackOff
kubectl delete pod broken crashy
```

Command	What it is for
`kubectl get <kind>`	List objects; add `-o wide`, `-o yaml`, `-w`, `-A` (all namespaces)
`kubectl describe <kind> <name>`	Human-readable details and Events - first stop for debugging
`kubectl logs <pod> [-c ctr] [-f] [--previous]`	Container logs; `--previous` shows the crashed instance
`kubectl exec -it <pod> -- sh`	Run a command inside a container
`kubectl port-forward`	Tunnel a local port to a pod or service
`kubectl apply / delete -f`	Create/update or delete from files
`kubectl explain <path>`	Built-in field documentation

Common mistake

- Creating bare pods for real apps. A bare pod is **not** recreated if its node dies - use a Deployment (Class 3).
- Expecting to change most fields of a running pod. Pod specs are largely immutable - delete and recreate, or let a Deployment do it.

Class 2 recap

- Pod = smallest unit; one IP shared by its containers.
- YAML = apiVersion, kind, metadata, spec.
- Init containers run first, in order; sidecars run alongside.
- Labels + selectors connect objects together.
- Debug flow: get -> describe -> logs -> exec.

Class 2 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a pod?

The smallest deployable unit in Kubernetes: a group of one or more containers that share a network namespace (one IP), can share volumes, and are always scheduled together on the same node.

Q2. Why does Kubernetes use pods instead of running containers directly?

Pods let tightly coupled containers share network and storage and be managed as one unit. They also give Kubernetes a runtime-neutral abstraction for scheduling, IP assignment and lifecycle.

Q3. Can a pod have more than one container? When would you do that?

Yes. Use it when containers must share resources and lifecycle - for example a sidecar log shipper, a service-mesh proxy, or an adapter. Unrelated services should be in separate pods so they scale independently.

Q4. How do containers in the same pod communicate?

Over `localhost` because they share the network namespace, and through shared volumes such as emptyDir. They must use different ports.

Q5. What is an init container?

A container that runs to completion before the app containers start. Multiple init containers run in order. Used to wait for dependencies, prepare files, or run database migrations.

Q6. What are sidecar, ambassador and adapter patterns?

Sidecar adds a helper feature (logging, proxy). Ambassador proxies the app's outbound connections. Adapter standardises the app's output (for example exposing metrics in Prometheus format).

Q7. What are the four required top-level fields in a manifest?

`apiVersion`, `kind`, `metadata` and `spec`. Kubernetes adds `status`, which reflects the actual state.

Q8. What is the difference between kubectl create and kubectl apply?

`create` is imperative and fails if the object exists. `apply` is declarative: it creates the object if missing or updates it to match the file, tracking the last applied configuration.

Q9. Explain the pod lifecycle phases.

Pending (accepted, not yet running), Running (on a node, at least one container running), Succeeded (all containers exited 0), Failed (at least one exited with error), and Unknown (node not reporting).

Q10. What is CrashLoopBackOff and how do you troubleshoot it?

The container keeps crashing and Kubernetes restarts it with growing delays. Check `kubectl logs <pod> --previous`, `kubectl describe pod` (exit code, OOMKilled, events), the command, environment variables, config files and probes.

Q11. What is ImagePullBackOff?

The kubelet cannot pull the image - usually a wrong name or tag, a private registry without imagePullSecrets, or no network access. `kubectl describe pod` shows the exact error.

Q12. What restart policies exist for pods?

Always (default; used by Deployments), OnFailure (restart only on non-zero exit; used by Jobs) and Never.

Q13. What are labels and selectors?

Labels are key-value tags on objects. Selectors query objects by labels - equality-based (`app=web`) or set-based (`env in (dev,qa)`). Deployments, Services and NetworkPolicies use selectors to find pods.

Q14. What is the difference between labels and annotations?

Labels identify and select objects and have size/format limits. Annotations store non-identifying metadata for tools and humans (build info, ingress settings) and cannot be used in selectors.

Q15. How do you get a shell inside a running container?

`kubectl exec -it <pod> -- sh` (or bash). For multi-container pods add `-c <container>`. If the image has no shell, use `kubectl debug` with an ephemeral container.

Q16. How do you view logs of a specific container or a previous crash?

`kubectl logs <pod> -c <container>`; add `-f` to follow and `--previous` to see the last terminated instance.

Q17. How can you generate YAML quickly without writing it by hand?

Use `--dry-run=client -o yaml`, e.g. `kubectl run web --image=nginx --dry-run=client -o yaml > pod.yaml`, and use `kubectl explain` to discover fields.

Q18. Are pods permanent? What happens if a node dies?

No, pods are ephemeral. A bare pod on a dead node is lost. Controllers like Deployments create replacement pods with new names and IPs, which is why we never run bare pods in production.

Q19. What is a static pod?

A pod managed directly by the kubelet from a manifest file on the node (usually /etc/kubernetes/manifests), not by the API server. Control-plane components in kubeadm clusters run as static pods.

Q20. What is kubectl port-forward used for?

It tunnels a local port to a pod or service, e.g. `kubectl port-forward pod/web 8080:80`, so you can test an app from your laptop without exposing it publicly.